

VSCode Java 개발환경 완성 강의

코드 작성 · Git 관리 · Maven 빌드 · 단위 테스트 · 정적 분석



JDK 21
& VSCode



Git
& GitHub



Maven
빌드



JUnit 5
TDD



정적 분석
Checkstyle

🕒 총 강의 시간: 3시간 | 수준: 입문 ~ 초급 | Windows / macOS / Linux 공통

도구: Eclipse Temurin JDK 21 · Maven · JUnit 5 · Checkstyle · SpotBugs · PMD

전체 강의 구성 (3시간)

파트	시간	제목	핵심 내용
1부	25분	VSCode + JDK 21 Java 개발환경 구축	설치, IntelliSense, 첫 빌드/디버그
2부	25분	Git + GitHub + VSCode Git UI	이력 관리, push/pull, GitLens
3부	20분	Maven 빌드 시스템	pom.xml, 의존성, 빌드 라이프사이클
—	5분	휴식	—
4부	25분	JUnit 5 + Test Runner 단위 테스트 구축	테스트 환경 구축, TDD 사이클
5부	25분	JUnit 5 작성법 + TDD 실습	어노테이션, Fixture, 실습
6부	30분	정적 분석 (Language Support · Checkstyle · SpotBugs · PMD)	경고 제거, 규칙, 자동화
7부	5분	마크다운 + 마무리 Q&A	README 작성, 전체 정리

사전 준비 — 설치 목록

Eclipse Temurin JDK 21

adoptium.net (LTS · 완전 무료)

Apache Maven 3.9+

maven.apache.org

Git

git-scm.com/downloads

VSCode

code.visualstudio.com/download

Extension Pack for Java

VSCode Extensions 검색 설치

Checkstyle for Java

VSCode Extensions 검색 설치

⚠ Oracle JDK는 상업 이용 시 유료입니다. 반드시 Eclipse Temurin (Adoptium) 또는 Microsoft Build of OpenJDK를 사용하세요.

C++ 대응: MinGW-w64 → JDK 21 | CMake → Maven | GoogleTest → JUnit 5 | clang-tidy → Checkstyle | cppcheck → SpotBugs/PMD

1

VSCode Java 개발환경 구축

JDK 21 · Extension Pack for Java · 첫 빌드 · 디버깅

🕒 00:00 ~ 00:25 (25분)

VSCode 인터페이스 구성

Activity Bar

사이드바 보기 전환
Explorer · Git · Extensions · Testing

Side Bar

Activity Bar 선택에 따라
파일 탐색기 · 소스 컨트롤 등 표
시

Editor Group

파일 편집 영역
탭·분할 편집 지원

Panel (하단)

터미널 · 디버그 콘솔
출력 · 문제 탭

Status Bar

현재 파일 정보
등

💡 Ctrl+Shift+P → Command Palette 로 모든 명령을 검색·실행할 수 있습니다

Eclipse Temurin JDK 21 설치 & 환경변수

- ① adoptium.net 에서 Temurin 21 (LTS) 다운로드 및 설치
- ② 시스템 환경변수 설정
 - JAVA_HOME = C:\Program Files\Eclipse Adoptium\jdk-21.x.x
 - Path 에 %JAVA_HOME%\bin 추가
- ③ 새 터미널에서 설치 확인

```

java -version
# openjdk 21.x.x  Eclipse Adoptium
javac -version
# javac 21.x.x

```

JDK 종류	라이선스	추천 대상
Eclipse Temurin (Adoptium)	완전 무료 (GPL2+CE)	✓ 강의·개인·상업 모두 추천
Microsoft Build of OpenJDK	완전 무료 (GPL2+CE)	✓ Windows Azure 환경
Amazon Corretto	완전 무료 (GPL2+CE)	✓ AWS 환경
Oracle JDK	상업 이용 유료	✗ 상업 프로젝트 사용 주의

VSCode settings.json 에 java.jdk.home 경로를 직접 지정하면 여러 JDK를 프로젝트별 전환 가능합니다

Eclipse Temurin JDK 21 설치 & 환경변수

Which Version of JDK Should I Use?

<https://whichjdk.com/>

Which Version of JDK Should I Use?

English 한국어



To build and run Java applications, a Java Compiler, Java Runtime Libraries, and a Virtual Machine are required that implement the Java Platform, Standard Edition ("Java SE") specification.

The [OpenJDK](#) is the open source reference implementation of the Java SE Specification, but it is only the source code. Binary distributions are provided by different vendors for a number of supported platforms. These distributions differ in licenses, commercial support, supported platforms, and update frequency.

This site gives independent, yet opinionated recommendations.

Eclipse Temurin JDK 21 설치 & 환경변수

<https://adoptium.net/>
<https://adoptium.net/temurin/releases>



Extension Pack for Java — 6개 확장 묶음

Language Support for Java

(Red Hat)

IntelliSense · 자동완성
실시간 오류 표시

Debugger for Java

(Microsoft)

중단점 · 변수 감시
스텝 실행

Test Runner for Java

(Microsoft)

JUnit 4/5 · TestNG
UI 탐색기

Maven for Java

(Microsoft)

pom.xml 관리
의존성 추가·빌드

Project Manager for Java

(Microsoft)

프로젝트 뷰
클래스·패키지 관리

IntelliCode

(Microsoft)

AI 기반 코드 완성
컨텍스트 추론



설치: Ctrl+Shift+X → 'Extension Pack for Java' 검색 → Install → Reload Window

첫 번째 Java 프로젝트 — 빌드 & 디버그

- ① Ctrl+Shift+P → 'Java: Create Java Project' 실행
- ② 'No build tools' 선택 후 프로젝트 폴더 지정
- ③ src/App.java 파일 자동 생성 확인
- ④ F5 → 'Java' 선택 → Hello, World! 출력 확인
- ⑤ F9 로 중단점 설정 후 변수 값 확인

```
public class App {  
    public static void main(String[] args) {  
        // F9 로 여기 중단점 설정  
        String msg = "Hello, Java!";  
        int count = 42;  
        System.out.println(msg);  
        System.out.println(count);  
    }  
}
```

자동 생성 파일	역할
.vscode/settings.json	java.project.referencedLibraries 등 프로젝트 설정
.vscode/launch.json	디버거 실행 구성 (mainClass, args 등)
src/App.java	자동 생성된 기본 클래스

VSCode Java 개발 UI 핵심 요소

Java Projects 뷰

패키지·클래스·의존성
트리 구조로 탐색

Problems 패널

컴파일 오류·경고
Ctrl+Shift+M 단축키

Testing 패널

JUnit 테스트 트리뷰
개별·전체 실행

Maven 사이드바

Lifecycle 목표 실행
의존성 트리 확인

디버그 패널

Variables · Watch
Call Stack 확인

단축키	기능
F5	디버그 시작 Shift+F5: 디버그 없이 실행
F9	중단점 설정/해제
F10 / F11	Step Over / Step Into
Ctrl+Shift+M	Problems 패널 (컴파일 오류 목록)

2

Maven 빌드 시스템

pom.xml · 라이프사이클 · 의존성 관리 · 멀티 모듈

🕒 00:50 ~ 01:10 (20분)

Maven이 필요한 이유 & 설치

javac *.java 만으로는

- 의존 라이브러리 관리 불가
- 멀티 파일 빌드 번거로움
- 테스트 자동화 연동 없음
- 팀 빌드 환경 통일 불가

→ Maven으로 해결

- ① maven.apache.org/download → Binary zip 다운로드
- ② 압축 해제 후 M2_HOME 환경변수 설정
- ③ Path 에 %M2_HOME%\bin 추가
- ④ 설치 확인



```
mvn -version # Apache Maven 3.9.x
```

Maven 핵심 개념	설명
pom.xml	프로젝트 설정 파일 (의존성·플러그인·빌드 정보)
Lifecycle	clean → compile → test → package → install → deploy
.m2/repository	로컬 의존성 캐시 (~/.m2/) — 한 번 다운로드 후 재사용
Archetype	프로젝트 템플릿 (mvn archetype:generate 으로 빠른 시작)

pom.xml 작성 & Maven 라이프사이클



```
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>my-app</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
  </properties>

  <dependencies>
    <!-- JUnit 5 -->
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.11.0</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
```

명령어	동작
mvn compile	소스 코드 컴파일
mvn test	단위 테스트 실행
mvn package	JAR/WAR 생성
mvn clean	target/ 폴더 정리
mvn install	로컬 저장소에 설치
mvn dependency:tree	의존성 트리 출력



휴식 — 5분

Java ☕ 한 잔의 여유 | 01:10 ~ 01:15

3

JUnit 5 + Test Runner 단위 테스트 구축

환경 설치 · pom.xml 설정 · TDD 사이클 개념

🕒 01:15 ~ 01:40 (25분)

단위 테스트 & TDD 개념

TDD 사이클



RED

실패하는 테스트 먼저 작성



GREEN

테스트 통과할 최소 코드 구현



REFACTOR

동작 유지하며 코드 정리

구성요소	역할
JUnit 5	Java 표준 단위 테스트 프레임워크
Test Runner for Java	VSCode Testing 패널 UI 연동
Maven Surefire	mvn test 시 JUnit 5 자동 실행
@Test	테스트 메서드 지정 어노테이션
@BeforeEach	각 테스트 전 실행 (C++ SetUp 대응)

💡 테스트 코드는 `src/test/java/` 폴더에 작성합니다. Maven은 이 폴더를 자동으로 테스트 소스로 인식합니다.

JUnit 5 환경 구축 — pom.xml & 프로젝트 구조

프로젝트 폴더 구조

```
my-app/  
├── src/  
│   ├── main/java/com/example/  
│   │   ├── Calculator.java  
│   │   └── StringUtils.java  
│   └── test/java/com/example/  
│       ├── CalculatorTest.java  
│       └── StringUtilsTest.java  
└── pom.xml
```

```
<!-- JUnit 5 의존성 -->  
<dependency>  
  <groupId>org.junit.jupiter</groupId>  
  <artifactId>junit-jupiter</artifactId>  
  <version>5.11.0</version>  
  <scope>test</scope>  
</dependency>  
  
<!-- Maven Surefire (JUnit 5 실행) -->  
<plugin>  
  <artifactId>maven-surefire-plugin</artifactId>  
  <version>3.5.0</version>  
</plugin>
```

💡 설치: Ctrl+Shift+X → 'Test Runner for Java' 는 Extension Pack for Java 에 포함 — Activity Bar의 Testing 아이콘으로 접근

4

JUnit 5 작성법 + TDD 실습

어노테이션 · Assertions · @BeforeEach · 실습 시나리오

🕒 01:40 ~ 02:05 (25분)

JUnit 5 주요 어노테이션 & Assertions

어노테이션	설명
@Test	테스트 메서드 지정
@BeforeEach	각 테스트 전 실행 (SetUp)
@AfterEach	각 테스트 후 실행 (TearDown)
@BeforeAll	클래스 전체 전 1회 실행
@AfterAll	클래스 전체 후 1회 실행
@DisplayName	테스트 이름 커스텀 지정
@Disabled	테스트 비활성화
@ParameterizedTest	매개변수 반복 테스트
@Nested	중첩 테스트 클래스

주요 Assertions

메서드	설명
assertEquals(expected, actual)	값 동등 검증
assertTrue(condition)	true 검증
assertFalse(condition)	false 검증
assertNotNull(object)	null 아님 검증
assertThrows(Type, lambda)	예외 발생 검증
assertAll(executables...)	복수 검증 일괄 실행
assertTimeout(duration, fn)	시간 제한 검증



GoogleTest의 EXPECT_ 계열 = JUnit Assertions | ASSERT_ 계열 = assertThrows 또는 assumeTrue

JUnit 5 테스트케이스 작성 예시

```

import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class CalculatorTest {
    private Calculator calc;

    @BeforeEach // 각 테스트 전 실행
    void setUp() { calc = new Calculator(); }

    @Test
    @DisplayName("덧셈 기본 검증")
    void testAdd() {
        assertEquals(5, calc.add(2, 3));
        assertEquals(0, calc.add(-1, 1));
    }

    @Test
    @DisplayName("0으로 나누면 예외")
    void testDivideByZero() {
        assertThrows(ArithmeticException.class,
            () -> calc.divide(10, 0)); }

```

C++ → Java 대응 비교

GoogleTest (C++)	JUnit 5 (Java)
TEST(Suite, Name)	@Test 메서드
EXPECT_EQ(a,b)	assertEquals(a,b)
EXPECT_TRUE(c)	assertTrue(c)
EXPECT_THROW	assertThrows(...)
TEST_F + SetUp	@BeforeEach
TearDown	@AfterEach



TDD: @Test 작성 → 빨간 실패 확인 → Calculator 구현 → 초록 통과 → 리팩토링

@ParameterizedTest — 매개변수 반복 테스트



```
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.*;

class StringUtilsTest {
    // 여러 입력을 자동으로 반복 실행
    @ParameterizedTest
    @ValueSource(strings = {"racecar", "radar", "level"})
    void testIsPalindrome(String word) {
        assertTrue(StringUtils.isPalindrome(word));
    }

    // 입력/기댓값 쌍으로 테스트
    @ParameterizedTest
    @CsvSource({"1,2,3", "0,0,0", "-1,1,0"})
    void testAdd(int a, int b, int expected) {
        assertEquals(expected,
            new Calculator().add(a, b));
    }
}
```

@ParameterizedTest 입력 소스 종류

어노테이션	설명
@ValueSource	단일 값 배열
@CsvSource	CSV 형식 다중 값
@CsvFileSource	CSV 파일에서 읽기
@MethodSource	정적 메서드 반환값
@EnumSource	Enum 상수 목록

💡 pom.xml 에 junit-jupiter-params 의존성 추가 필요: groupId org.junit.jupiter / artifactId junit-jupiter-params

5

정적 분석 — Language Support · Checkstyle · SpotBugs · PMD

빌드 없이 코드 품질을 높이는 자동화 도구

🕒 02:05 ~ 02:35 (30분)

Java 정적 분석 도구 개요

프로그램을 실제로 실행하지 않고 소스코드를 분석하여 버그·보안 취약점·코딩 규칙 위반을 찾아냅니다.

Language Support for Java (Red Hat)

IntelliSense 기반
실시간 경고 표시
Extension Pack 내장

에디터 저장 시 실시간

Checkstyle

코딩 규칙 검사
네이밍·포맷·관행
VSCode 확장 지원

저장/커밋 전 검사

SpotBugs

바이트코드 분석
null·메모리·동시성
Maven 플러그인

mvn 빌드 자동화

PMD

소스코드 분석
미사용변수·복잡도
Maven 플러그인

mvn 빌드 자동화

 4가지 도구를 함께 사용하면 레이어드 방어(layered defense) 효과 — 실시간 → 규칙 → 버그 패턴 → 코드 품질까지 완전 커버

Language Support for Java — 실시간 분석

- Extension Pack for Java 설치만으로 자동 활성화
- 저장 시 빨간/노란 물결선으로 즉시 오류·경고 표시
- Problems 패널 (Ctrl+Shift+M) 에서 전체 목록 확인
- Quick Fix (Ctrl+.) 로 자동 수정 제안 적용

settings.json 설정	설명
"java.errors.incompleteClasspath.severity"	미완성 클래스패스 심각도
"java.compile.nullAnalysis.mode"	null 분석 활성화
"editor.formatOnSave": true	저장 시 자동 포맷
"java.saveActions.organizeImports"	저장 시 import 정리

탐지 가능한 주요 문제

Null Pointer 위험

미사용 import 경고

타입 불일치

접근 제한자 오류

Override 누락

Deprecated API 사용



Ctrl+Shift+M → Problems 탭에서 프로젝트 전체 오류/경고를 한눈에 확인할 수 있습니다

Checkstyle — 설치 & VSCode 연동

- ① VSCode: Ctrl+Shift+X → 'Checkstyle for Java' 설치
- (shengchen.vscode-checkstyle)
- ② Ctrl+Shift+P → 'Checkstyle: Set Checkstyle Configuration File'
- ③ Google Style 또는 Sun Style 기본 제공 선택
- ④ 또는 프로젝트 루트에 checkstyle.xml 직접 작성

settings.json 설정

```
{
  "java.checkstyle.configuration":
    "${workspaceFolder}/checkstyle.xml",
  "java.checkstyle.version": "10.18.1",
  "java.checkstyle.modules": ["Checker"]
}
```

checkstyle.xml 주요 규칙

```
<module name="Checker">
  <module name="TreeWalker">
    <!-- 중괄호 필수 -->
    <module name="NeedBraces"/>
    <!-- equals와 hashCode 쌍 검증 -->
    <module name="EqualsHashCode"/>
    <!-- 매직 넘버 사용 금지 -->
    <module name="MagicNumber"/>
    <!-- 순환 복잡도 10 이하 -->
    <module name="CyclomaticComplexity"/>
    <!-- 불필요한 빈 블록 -->
    <module name="EmptyBlock"/>
  </module>
</module>
```

💡 Google Java Style은 구글이 공식 배포하는 checkstyle-google.xml을 그대로 사용 가능합니다.

Checkstyle — 주요 경고 유형 및 수정 예시

NeedBraces

Before: `if (x > 0) return x;`

After: `if (x > 0) { return x; }`

MagicNumber

Before: `double tax = price * 0.1;`

After: `final double TAX = 0.1;
double tax = price * TAX;`

EqualsHashCode

Before: `// equals()만 구현, hashCode() 없음`

After: `// equals()와 hashCode() 함께 구현`

EmptyBlock

Before: `} catch (Exception e) {}`

After: `} catch (Exception e) { log(e); }`

ConstantName

Before: `static final int maxSize = 100;`

After: `static final int MAX_SIZE = 100;`

SpotBugs — 바이트코드 버그 패턴 분석

- ① pom.xml에 SpotBugs Maven Plugin 추가
- ② mvn spotbugs:check 실행 → 버그 리포트 생성
- ③ target/spotbugsXml.xml 또는 HTML 리포트 확인

SpotBugs 주요 버그 카테고리

카테고리	탐지 내용
NP (Null)	null 역참조 위험
RCN	불필요한 null 검사
OS (Resources)	스트림.파일 미닫힘
SQL	SQL 인젝션 위험
DM	박싱/언박싱 비효율
IS (동시성)	동기화 문제
BC (캐스팅)	잘못된 형변환

```

<!-- pom.xml -->
<plugin>
  <groupId>com.github.spotbugs</groupId>
  <artifactId>spotbugs-maven-plugin</artifactId>
  <version>4.8.6.4</version>
  <configuration>
    <effort>Max</effort>
    <threshold>Low</threshold>
  </configuration>

```

```

mvn spotbugs:check    # 빌드 실패 처리    |    mvn spotbugs:gui    # GUI 뷰어로 확인

```

PMD — 소스코드 품질 분석

- ① pom.xml에 Maven PMD Plugin 추가
- ② mvn pmd:check 실행 → 소스코드 품질 분석
- ③ target/site/pmd.html 리포트 확인

PMD 주요 규칙셋

규칙셋	검사 내용
bestpractices.xml	Java 모범 관행
errorprone.xml	오류 가능성 코드
performance.xml	성능 비효율 코드
design.xml	설계 복잡도
multithreading.xml	동시성 문제
security.xml	보안 취약점

```

<!-- pom.xml -->
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-pmd-plugin</artifactId>
  <version>3.25.0</version>
  <configuration>
    <rulesets>

```

```

mvn pmd:check      # PMD 분석 (위반 시 빌드 실패)      |   mvn pmd:pmd    # 리포트만 생성
</configuration>

```

💡 PMD CPD(중복 코드 검출): mvn pmd:cpd 로 복사-붙여넣기 코드를 탐지할 수 있습니다.

정적 분석 도구 비교 & 통합 전략

	Language Support	Checkstyle	SpotBugs	PMD
분석 대상	소스코드	소스코드	바이트코드(.class)	소스코드
실행 시점	실시간 (타이핑)	저장/커밋 전	mvn 빌드 시	mvn 빌드 시
강점	즉각 피드백	코딩 규칙 일관성	런타임 버그 탐지	코드 품질·복잡도
VSCode 통합	자동 내장	확장 설치	Maven 플러그인	Maven 플러그인
설치 난이도	★ (자동)	★★ (확장+설정)	★★ (pom.xml)	★★ (pom.xml)
C++ 대응	C/C++ Extension	clang-tidy	cppcheck	clang-tidy PMD

권장 통합 전략

코딩 중: Language Support 실시간 경고 확인 → **커밋 전:** Checkstyle 규칙 검사 → **CI/CD:** SpotBugs + PMD 자동 분석

💡 mvn verify 한 번으로 compile → test → SpotBugs → PMD → Checkstyle 을 순서대로 실행할 수 있습니다.

pom.xml — 정적 분석 전체 통합 설정

① Checkstyle ② SpotBugs

```

<!-- ① Checkstyle: 코딩 규칙 -->
<plugin>
  <artifactId>
    maven-checkstyle-plugin</artifactId>
  <version>3.5.0</version>
  <configuration>
    <configLocation>
      checkstyle.xml</configLocation>
    <failOnViolation>true</failOnViolation>
  </configuration>
</plugin>
<!-- ② SpotBugs: 버그 패턴 -->
<plugin>
  <groupId>com.github.spotbugs</groupId>
  <artifactId>
    spotbugs-maven-plugin</artifactId>
  <version>4.8.6.4</version>
</plugin>

```

③ PMD + 실행 방법

```

<!-- ③ PMD -->
<plugin>
  <artifactId>maven-pmd-plugin</artifactId>
  <version>3.25.0</version>
</plugin>

```

```

# 전체 분석 한 번에
mvn verify
# 개별 실행
mvn checkstyle:check spotbugs:check pmd:check

```

전체 분석 워크플로우

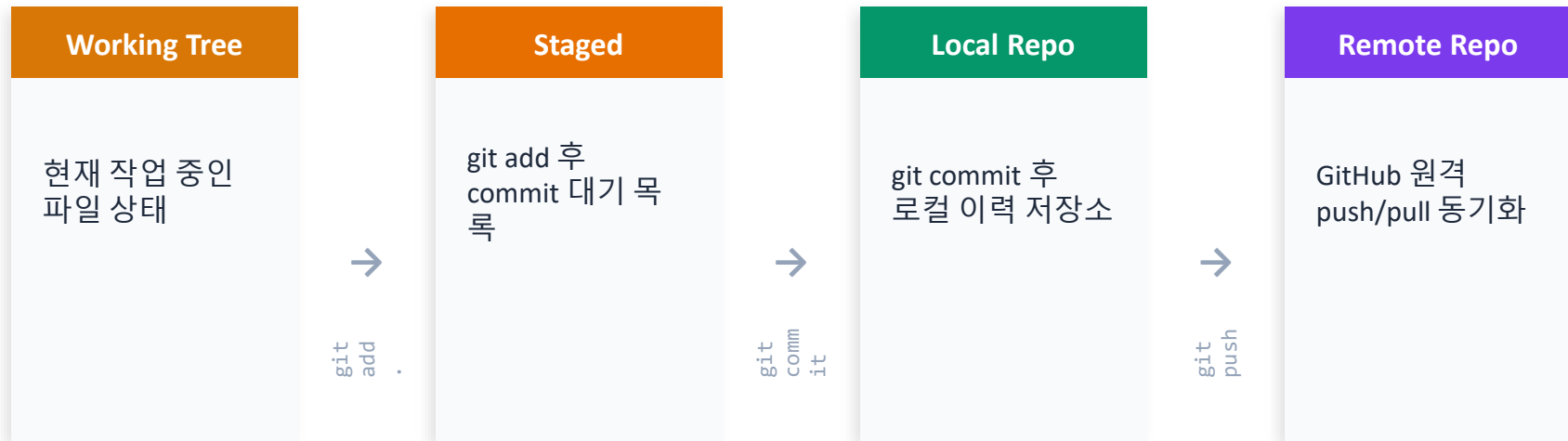
- 1 코딩 중 — Language Support 실시간
- 2 저장 시 — Checkstyle 규칙 검사
- 3 빌드 시 — SpotBugs 바이트코드 분석
- 4 CI/CD — PMD + mvn verify 자동화

6

init · add · commit · push · pull · GitLens

🕒 00:25 ~ 00:50 (25분)

Git 핵심 개념 & 기본 명령어



명령어	설명
<code>git init</code>	현재 폴더를 Git 관리 대상으로 초기화
<code>git add . / git status</code>	변경 파일 staged 전환 / 상태 확인
<code>git commit -m "메시지"</code>	staged 파일을 이력에 기록
<code>git log --oneline</code>	커밋 이력 요약 출력

VSCode Git UI + GitHub 연동

- ① Activity Bar → Source Control (Ctrl+Shift+G)
- ② Initialize Repository → git 초기화
- ③ 변경 파일 옆 + 버튼 → Staged Changes
- ④ 메시지 입력 후 Commit ✓ 클릭
- ⑤ ... 메뉴 → Undo Last Commit (커밋 취소)



```
# GitHub 원격 연결
git remote add origin
https://github.com/user/repo.git

# 첫 pull (이력 병합)
git pull origin main --allow-unrelated-histories

# push (upstream 설정)
git push --set-upstream origin main
```

.gitignore 권장 내용 (Java)



```
target/           # Maven 빌드 산출물
*.class
.classpath
.project
.settings/
```



GitLens 확장 설치 시 커밋 그래프·인라인 Blame·파일 히스토리를 시각적으로 확인할 수 있습니다. GitHub 연동 후 --set-upstream 사용 시 이후로는 git push 만으로 반영됩니다.

7

마크다운 + 마무리 Q&A

README 작성법 · 전체 워크플로우 정리

🕒 02:35 ~ 02:40 (5분)

마크다운 & README 작성법

문법	결과
# H1 / ## H2 / ### H3	헤더 (H1/H2/H3)
굵게 / *기울임*	굵게 / 기울임
[링크](URL)	하이퍼링크
![대체텍스트](이미지URL)	이미지 삽입
`인라인` / ```블록```	코드 강조
> 인용	인용문
- 항목 / 1. 항목	비순서 / 순서 목록
col1 col2	표 작성

README.md 권장 구성

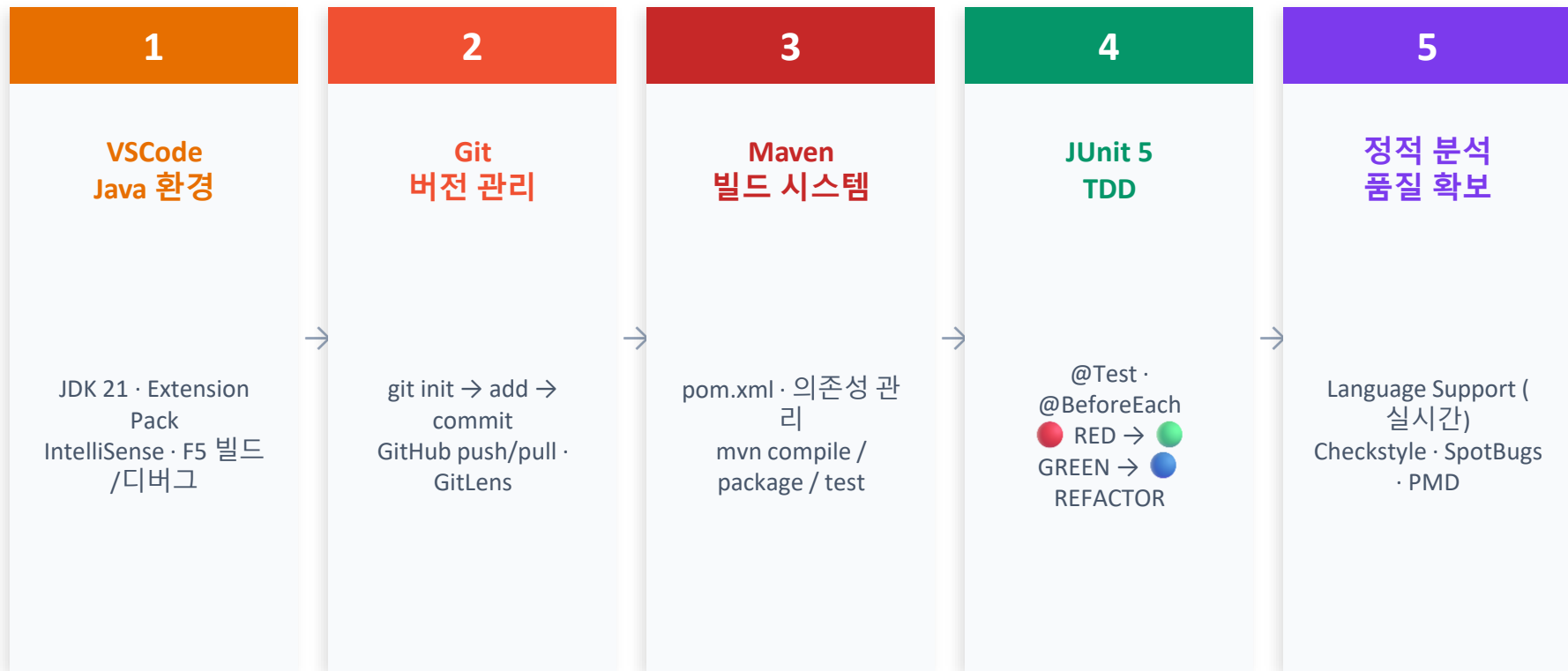
- # 프로젝트명
- ## 개요 — 프로젝트 목적
- ## 빌드 방법 — mvn compile / package
- ## 테스트 실행 — mvn test
- ## 정적 분석 — mvn verify
- ## 의존성 — JDK 21, Maven 3.9+

.gitignore 권장 (Java)



```
target/    *.class  *.jar
.classpath .project .settings/
.idea/    *.iml
```

전체 개발 워크플로우 정리



핵심 단축키 & 참고 자료

단축키	기능
F5	디버그 시작
Shift+F5	실행 (디버그 없이)
F9	중단점 설정/해제
F10 / F11	Step Over / Step Into
Ctrl+Shift+P	Command Palette
Ctrl+Shift+X	Extensions 열기
Ctrl+Shift+G	Source Control (Git)
Ctrl+Shift+M	Problems 탭 (정적 분석)
Ctrl+.	Quick Fix 제안
Ctrl+Shift+B	빌드 태스크 실행

참고 자료

VSCode Java 공식 문서

code.visualstudio.com/docs/languages/java

Eclipse Temurin JDK 21

adoptium.net

JUnit 5 공식 문서

junit.org/junit5/docs/current/user-guide

Checkstyle 공식

checkstyle.sourceforge.io

SpotBugs 공식

spotbugs.github.io

Apache Maven 공식

maven.apache.org/guides

수고하셨습니다! 🍵 🎉

Clean Code That Works!!!

- ✓ VSCode + Eclipse Temurin JDK 21 Java 개발환경
- ✓ Git + GitHub 소스 이력 관리
- ✓ Maven 빌드 시스템 (pom.xml · 라이프사이클 · 의존성)
- ✓ JUnit 5 + Test Runner TDD (@Test · @ParameterizedTest · Assertions)
- ✓ Language Support · Checkstyle · SpotBugs · PMD 정적 분석

모든 도구 100% 무료 오픈소스 · Windows / macOS / Linux 공통 적용 가능